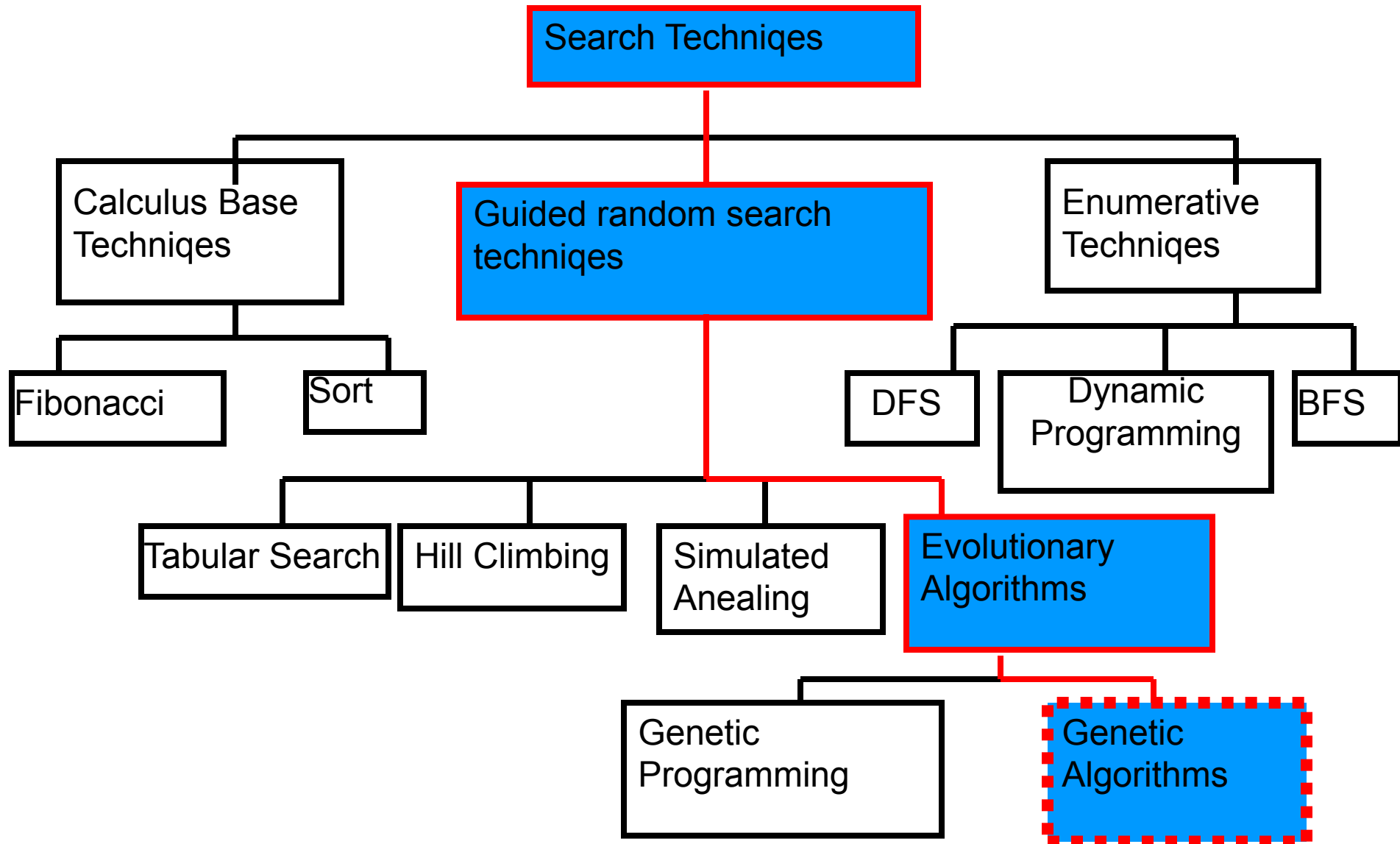


GENETIC ALGORITHM

Classes of Search Techniques



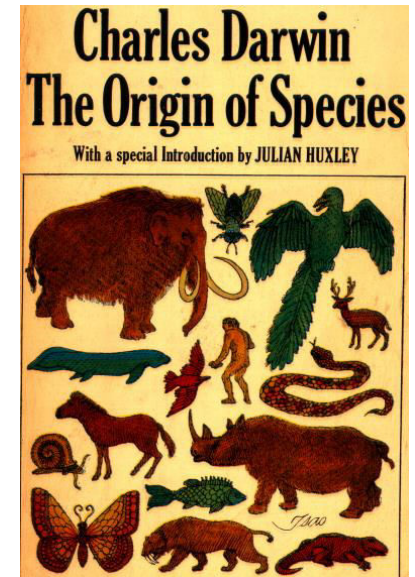
APPLICATIONS OF GENETIC ALGORITHMS

- **Optimization** - Genetic Algorithms are most commonly used in optimization problems wherein we have to maximize or minimize a given objective function value under a given set of constraints. The approach to solve Optimization problems has been highlighted throughout the tutorial.
- **Economics** - GAs are also used to characterize various economic models like the cobweb model, game theory equilibrium resolution, asset pricing, etc.
- **Neural Networks** - GAs are also used to train neural networks, particularly recurrent neural networks.
- **Parallelization** - GAs also have very good parallel capabilities, and prove to be very effective means in solving certain problems, and also provide a good area for research.

- **Image Processing** – GAs are used for various digital image processing (DIP) tasks as well like dense pixel matching.
- **Vehicle routing problems** - With multiple soft time windows, multiple depots and a heterogeneous fleet.
- **Scheduling applications** – GAs are used to solve various scheduling problems as well, particularly the time tabling problem.
- **Machine Learning** – as already discussed, genetics based machine learning (GBML) is a niche area in machine learning.
- **Robot Trajectory Generation** – GAs have been used to plan the path which a robot arm takes by moving from one point to another.
- **Parametric Design of Aircraft** – GAs have been used to design aircrafts by varying the parameters and evolving better solutions.
- **DNA Analysis** – GAs have been used to determine the structure of DNA using spectrometric data about the sample.
- **Multimodal Optimization** – GAs are obviously very good approaches for multimodal optimization in which we have to find multiple optimum solutions.
- **Traveling salesman problem and its applications** – GAs have been used to solve the TSP, which is a well-known combinatorial problem using novel crossover and packing strategies.

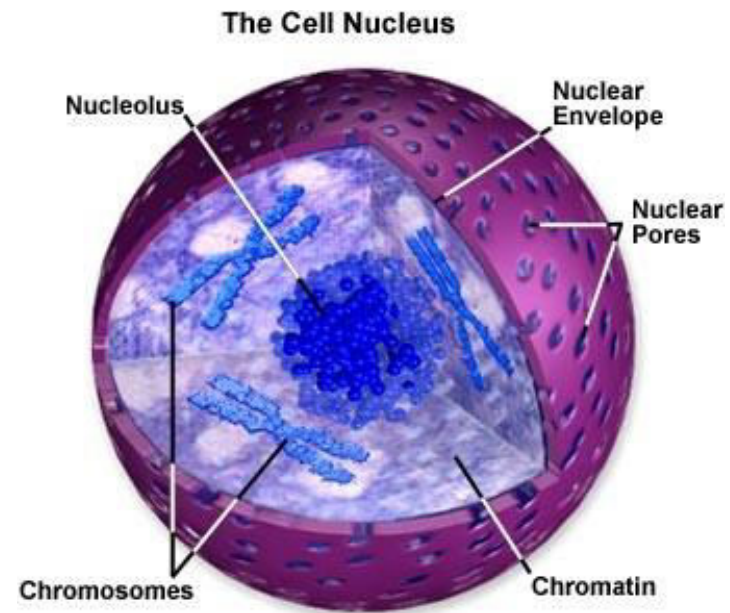
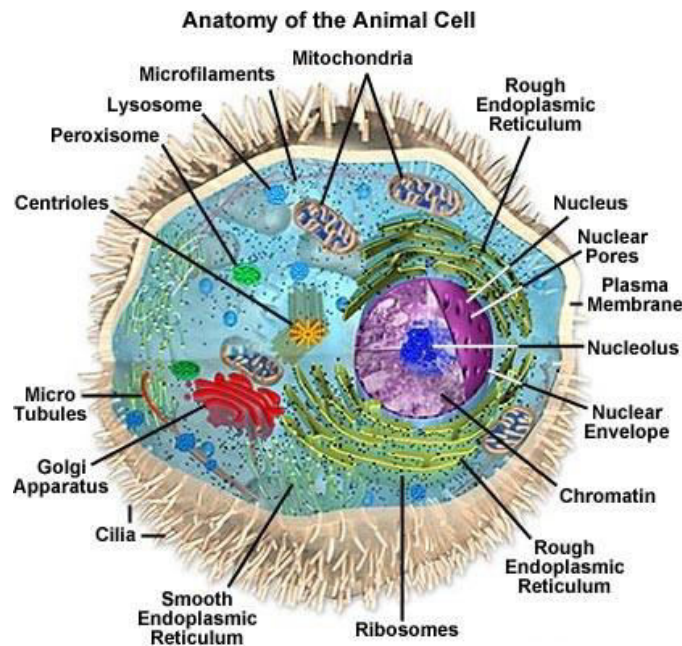
General Introduction to GAs

- Genetic algorithms (GAs) are a technique to solve problems which need optimization.
- GAs are a subclass of **Evolutionary Computing** and are random search algorithms.
- GAs are based on Darwin's theory of evolution.
- History of GAs:
 - Evolutionary computing evolved in the 1960s.
 - GAs were created by John Holland in the mid-1970s.



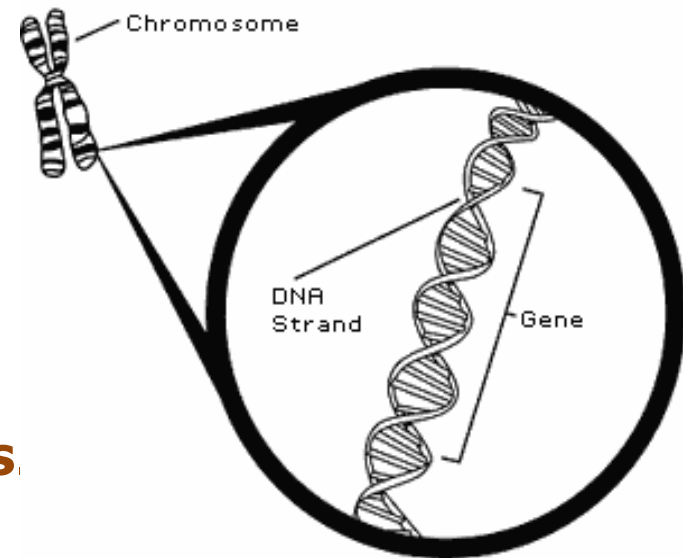
Biological Background – The Cell

- Every cell is a complex of many small “factories”
- working together.
- The center of this all is the **cell nucleus**.
- The nucleus contains the genetic information.



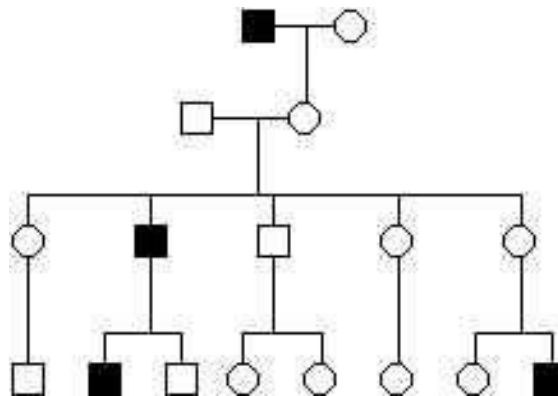
Biological Background– Chromosomes

- Genetic information is stored in the **chromosomes**.
- Each chromosome is build of **DNA**.
- Chromosomes in humans form pairs.
- There are 23 pairs.
- The chromosome is divided in parts: **genes**.
- Genes code for properties.
- The possibilities of the genes for one property is called: **allele**.
- Every gene has an unique position on the chromosome: **locus**.



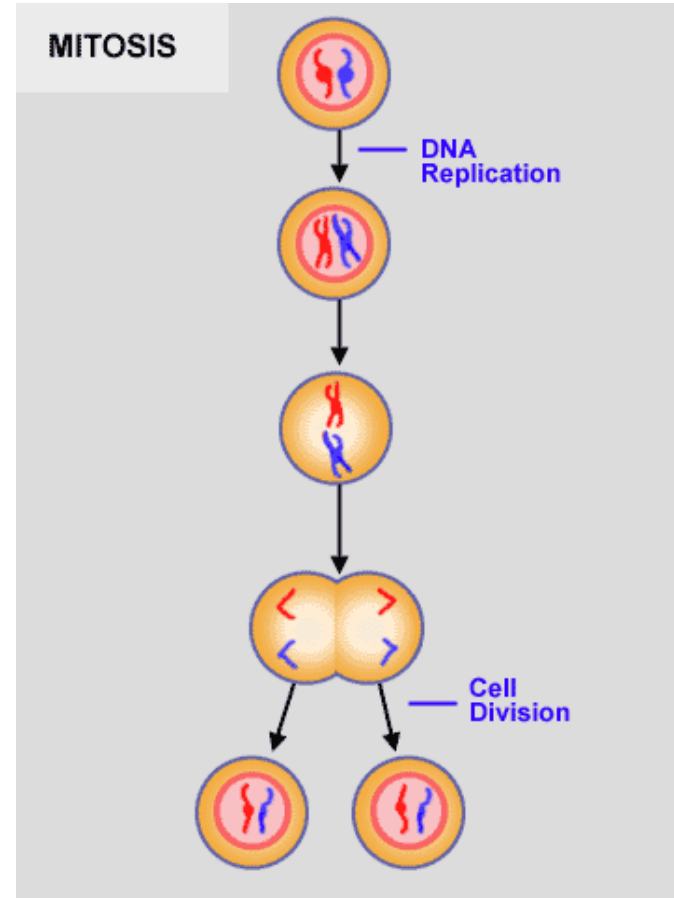
Biological Background– Genetics

- The entire combination of genes is called **genotype**.
- A genotype develops into a **phenotype**.
- **Alleles** can be either dominant or recessive.
- Dominant alleles will always express from the genotype to the phenotype.
- Recessive alleles can survive in the population for many generations without being expressed.



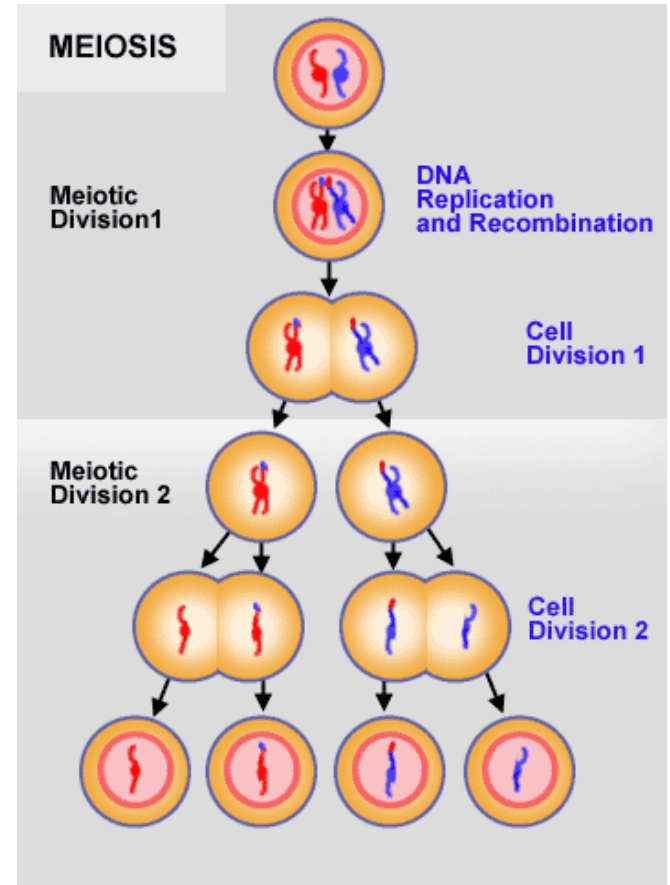
Biological Background– Reproduction

- Reproduction of genetical information:
 - Mitosis,
 - Meiosis.
- Mitosis is copying the same genetic information to new offspring: there is no exchange of information.
- Mitosis is the normal way of growing of multicell structures, like organs.



Biological Background– Reproduction

- After meiotic division 2 **gametes** appear in the process.
- In reproduction two gametes conjugate to a **zygote** which will become the new individual.
- Hence genetic information is shared between the parents in order to create new offspring.

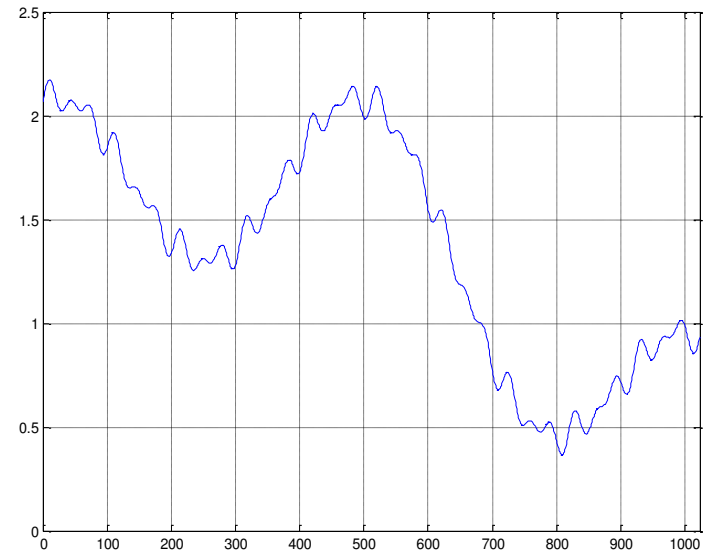


Biological Background (6) – Natural Selection

- The origin of species: “Preservation of favorable variations and rejection of unfavorable variations.”
- There are more individuals born than can survive, so there is a continuous **struggle for life**.
- Individuals with an advantage have a greater chance for survive: **survival of the fittest**. For example, Giraffes with long necks.
- Genetic variations due to crossover and mutation.

Genetic Algorithm (1) – Search Space

- Most often one is looking for the best solution in a specific subset of solutions.
- This subset is called the **search space** (or state space).
- Every point in the search space is a possible solution.
- Therefore every point has a **fitness** value, depending on the problem definition.
- GAs are used to search the search space for the best solution, e.g. a minimum.
- Difficulties are the local minima and the starting point of the search.

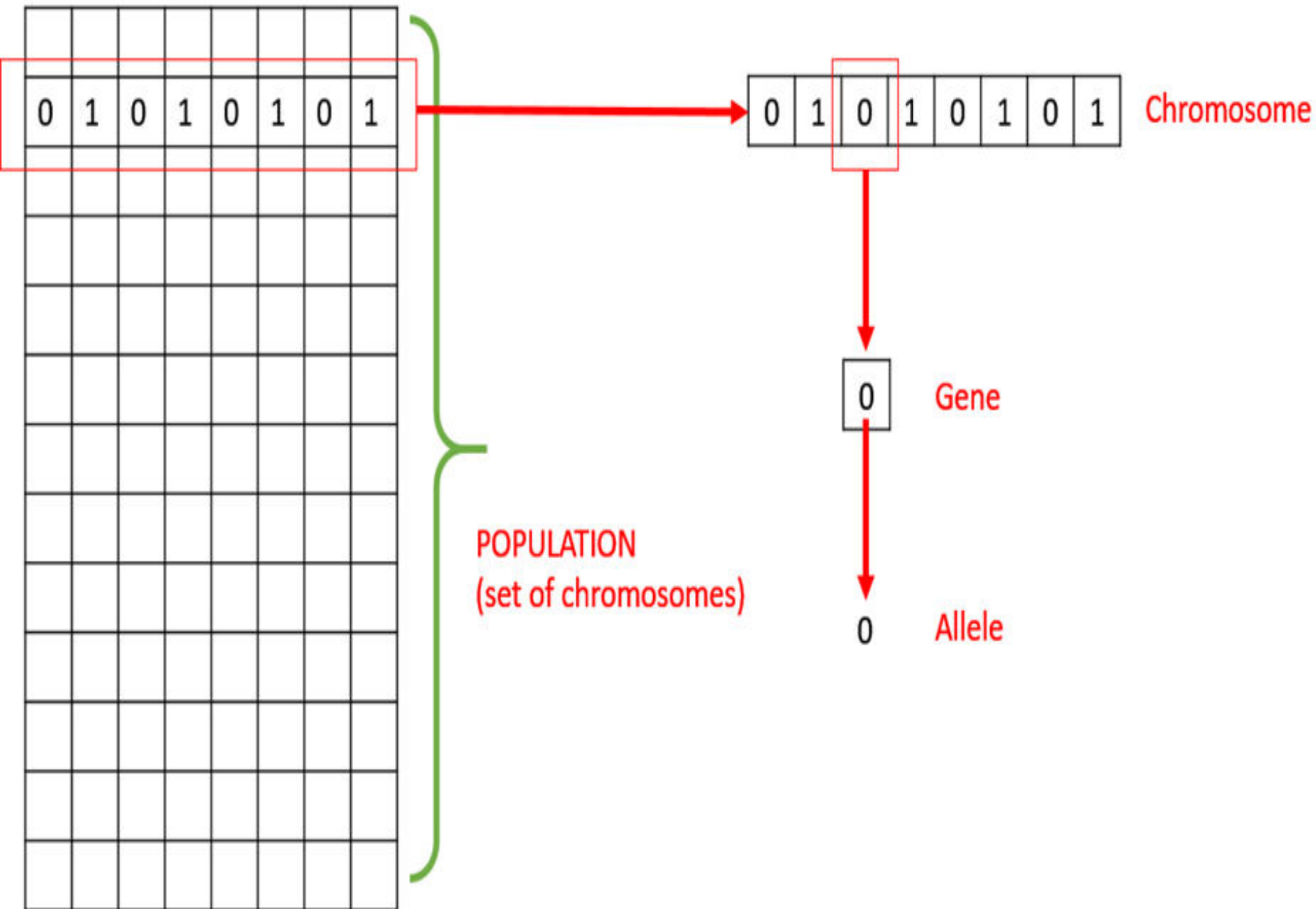


Genetic Algorithm– Basic Algorithm

- Starting with a subset of n randomly chosen solutions from the search space (i.e. chromosomes).
This is the **population**.
- This population is used to produce a next **generation** of individuals by reproduction.
- Individuals with a higher **fitness** have more chance to reproduce (i.e. natural selection).

Comparison of Natural and GA Terminology

Natural	Genetic Algorithm
Chromosome	String
Gene	Feature or character
Allele	Feature value
Locus	String position
Genotype	Structure
Phenotype	Parameter set, a decoded structure



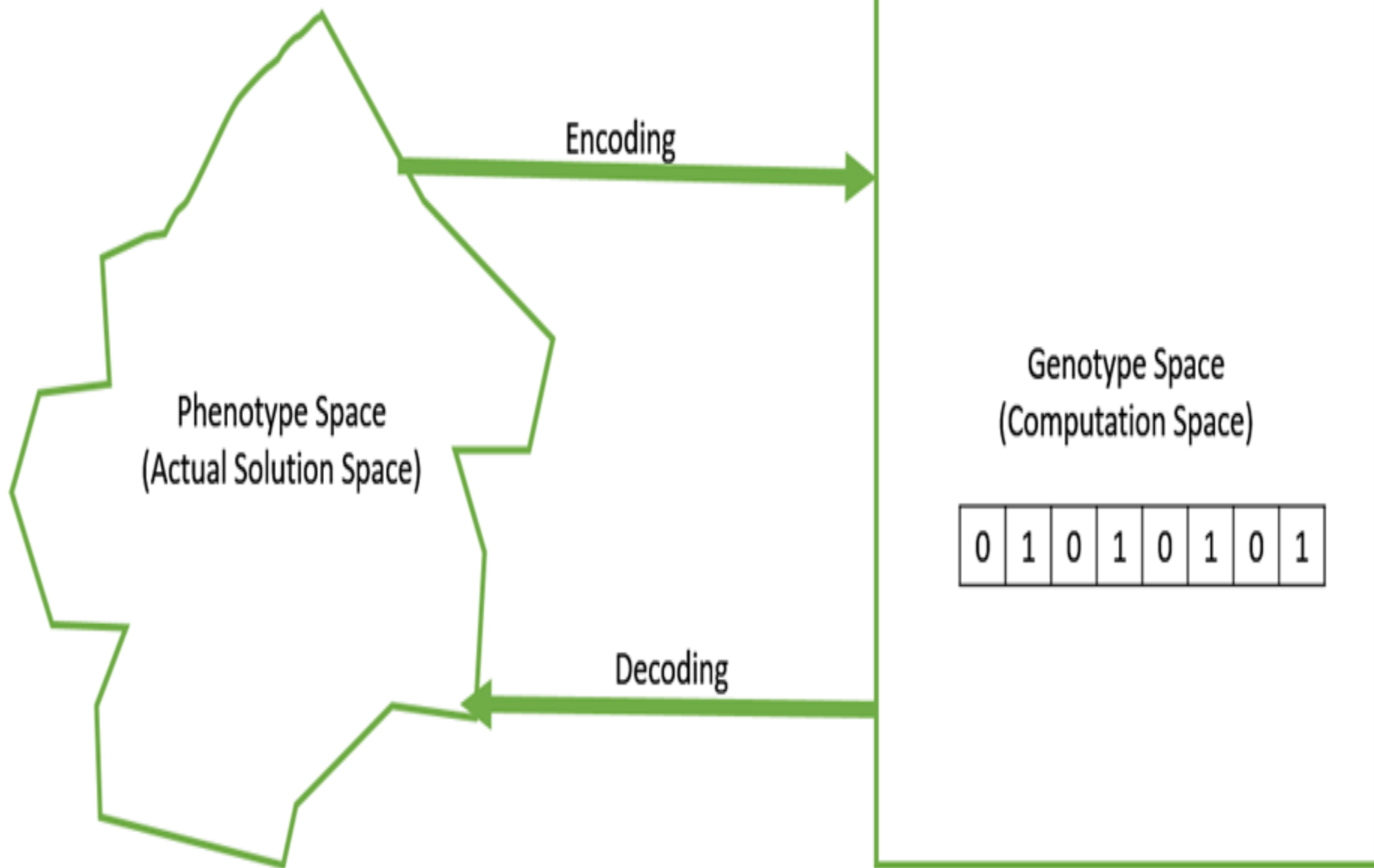
Phenotype Space
(Actual Solution Space)

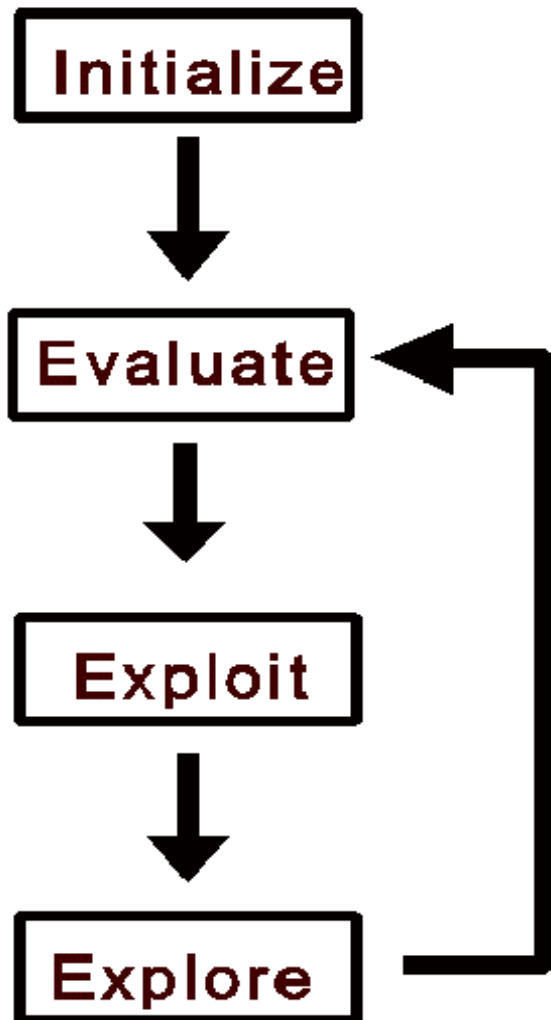
Encoding

Genotype Space
(Computation Space)

0	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---

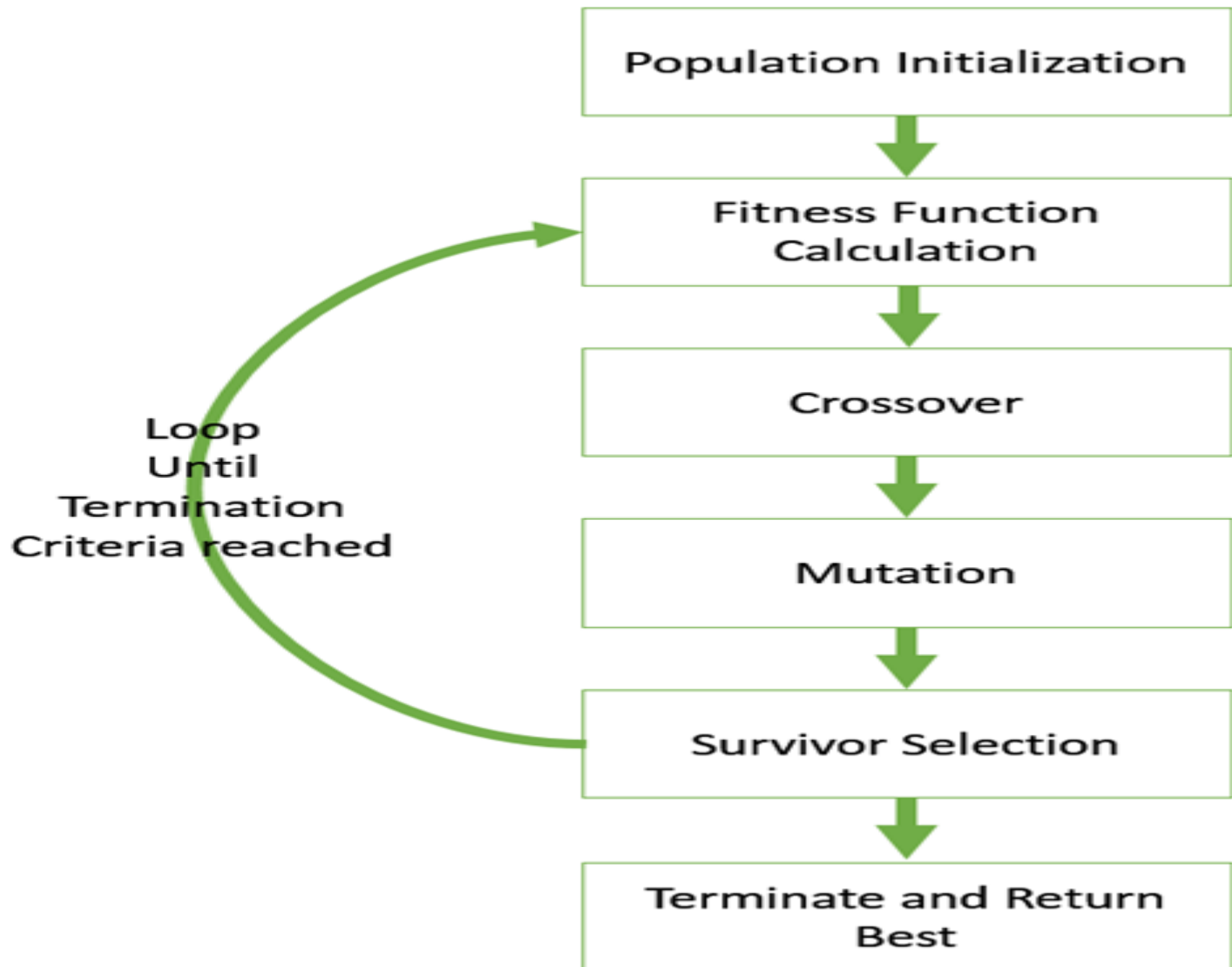
Decoding





- All individuals in population evaluated by fitness function.
- Individuals allowed to reproduce (selection), crossover, mutate.

Flowchart of GA



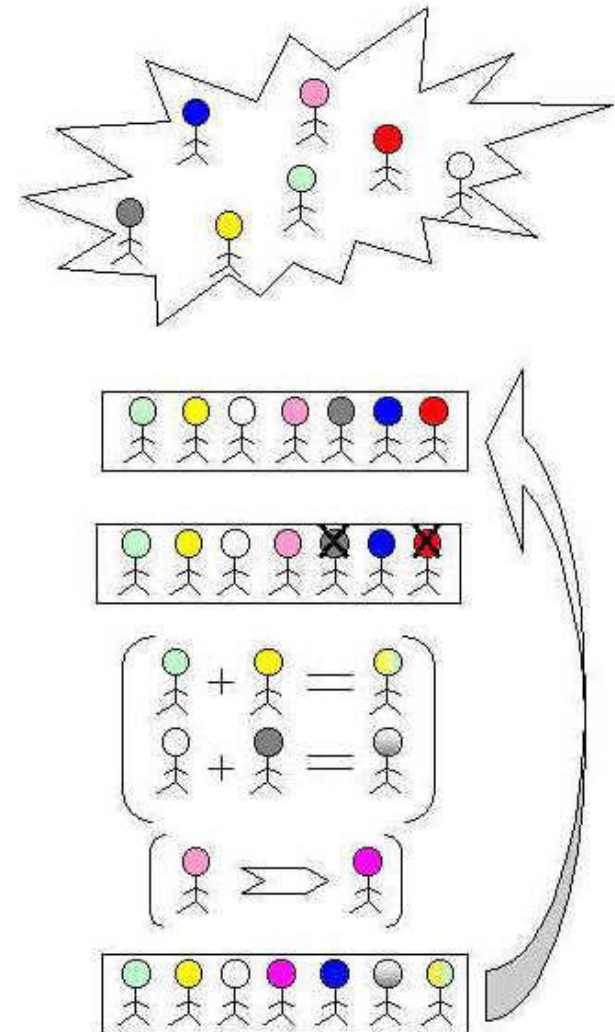
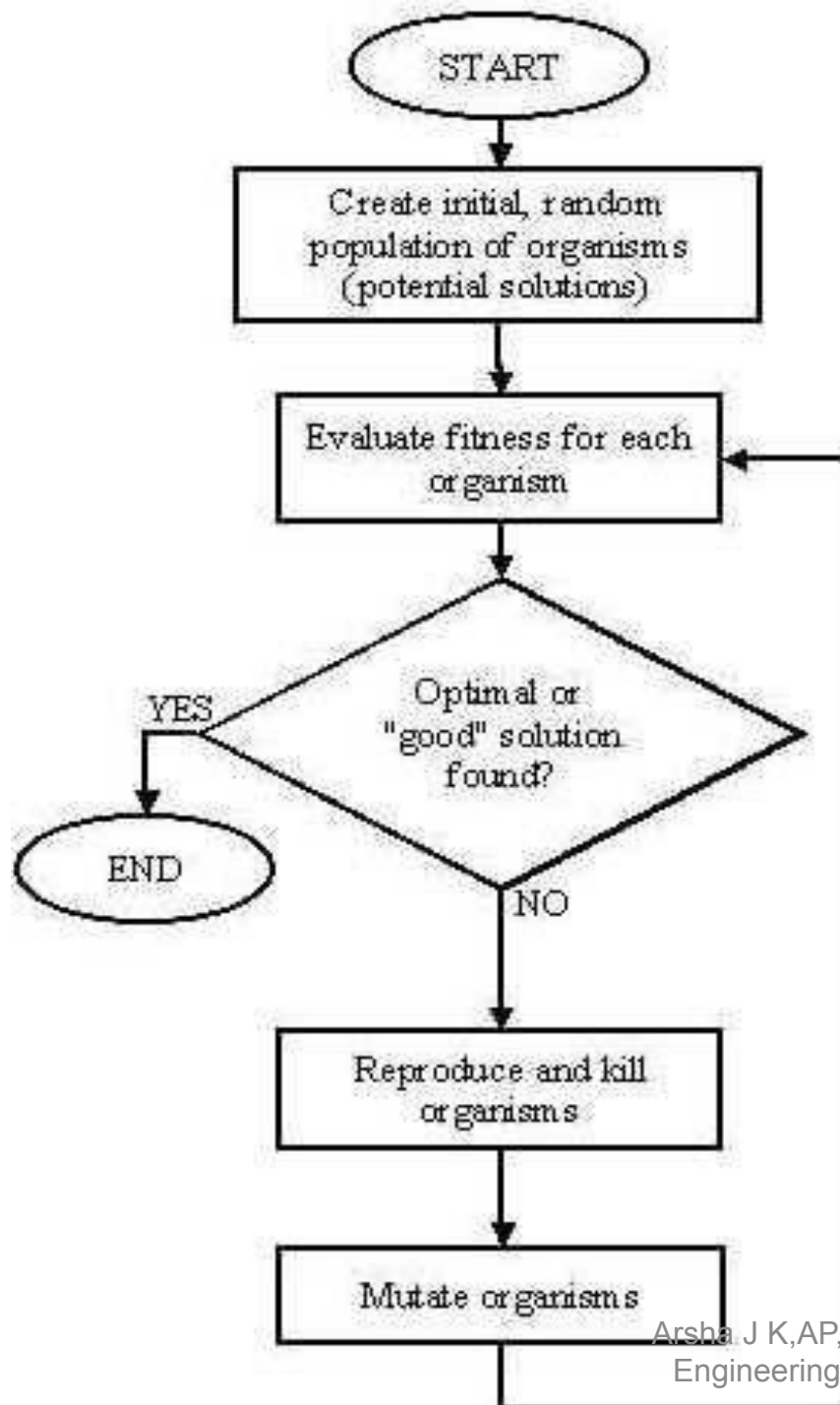
Genetic Algorithm (3) – Basic Algorithm

Outline of the basic algorithm

- 0 START** : Create random population of n chromosomes
- 1 FITNESS** : Evaluate fitness $f(x)$ of each chromosome in the population
- 2 NEW POPULATION**
 - 1 REPRODUCTION/SELECTION** : Based on $f(x)$
 - 2 CROSS OVER** : Cross-over chromosomes
 - 3 MUTATION** : Mutate chromosomes
- 3 REPLACE** : Replace old with new population: the new generation
- 4 TEST** : Test problem criterium
- 5 LOOP** : Continue step 1 – 4 untill criterium is satisfied

A generalized pseudo-code for a GA is explained in the following program:

```
GA()  
  initialize population  
  find fitness of population  
  while (termination criteria is reached) do  
    parent selection  
    crossover with probability pc  
    mutation with probability pm  
    decode and fitness calculation  
    survivor selection  
    find best  
  return best
```



OPERATORS IN GA

- Coding
- Selection
- Crossover/Recombination
- Mutation

ENCODING/CODING

- Process of representing individual gene
- Depends mainly on solving the problem
- Different coding techniques are
 - Binary encoding
 - Octal encoding
 - Hexadecimal encoding
 - Permutation/Real number Encoding
 - Value encoding
 - Tree encoding

Binary Encoding

- Each chromosome encodes a binary string
- Each bit represent some characteristics

CHROMOSOME 1	110100011010
CHROMOSOME 2	110111111010

Octal Encoding

- Each chromosome encodes an octal string
- From 0- 7

CHROMOSOME 1	03467216
CHROMOSOME 2	72561423

Hexadecimal Encoding

- Each chromosome encodes a hexadecimal string
- From 0- 9,A-F

CHROMOSOME 1	9CE7
CHROMOSOME 2	3DBA

Permutation Encoding

- Each chromosome encodes a string of numbers represented in sequence
- Only useful for ordering problem

CHROMOSOME 1	1 5 3 2 6 4 7 9 8
CHROMOSOME 2	8 5 6 7 2 3 1 4 9

Value Encoding

- Each chromosome encodes a string of values and values can be anything concerning the problem

CHROMOSOME 1	1.2354 3.5687 4.7894 5.897461
CHROMOSOME 2	ASDFGERTYUILKJGF
CHROMOSOME 3	(back),(back),(forward), (right), (left)

Tree Encoding

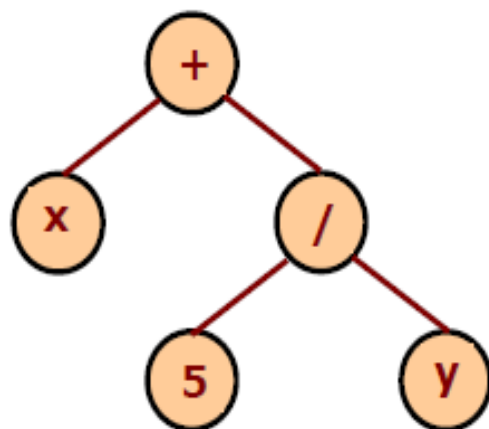
Tree encoding is used mainly for evolving programs or expressions.

For genetic programming :

- In tree encoding, every chromosome is a tree of some objects, such as functions or commands in programming language.
- Tree encoding is useful for evolving programs or any other structures that can be encoded in trees.
- The crossover and mutation can be done relatively easy way .

Example :

Chromosome A



$(+ x (/ 5 y))$

Chromosome B



$(do_untill\ step\ wall)$

SELECTION

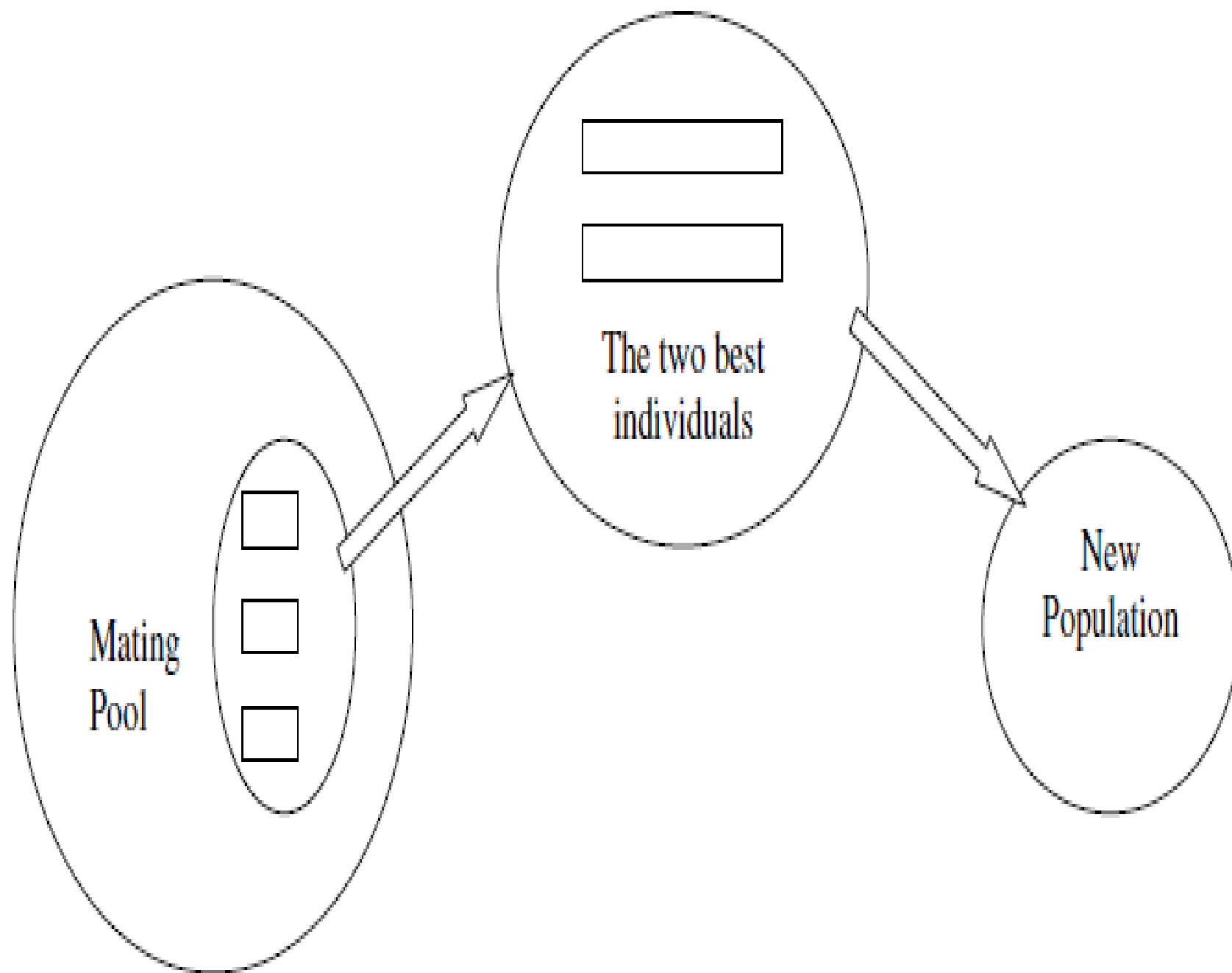
- **Selection is the process of choosing 2 parents from population for crossing**
- **After deciding an encoding next step is to decide how to perform selection**
- **Selection is the method that randomly pick chromosomes out of the population according to their evaluation function**
- **The higher the fitness function value , the better chance that an individual will be selected**

SELECTION

- The **selection pressure** is defined as the degree to which the better individuals are favored
- The higher the selection pressure, the more the individuals are favored
- The **convergence rate** of GA is largely determined by the magnitude of the selection pressure
- Higher selection pressure resulting in the higher convergence rate
- If the selection pressure is too low, the convergence rate will be slow and the GA will take unnecessarily longer to find the optimal solution
- If the selection pressure is too high, there is an increased change of the GA prematurely converging to an incorrect solution

SELECTION

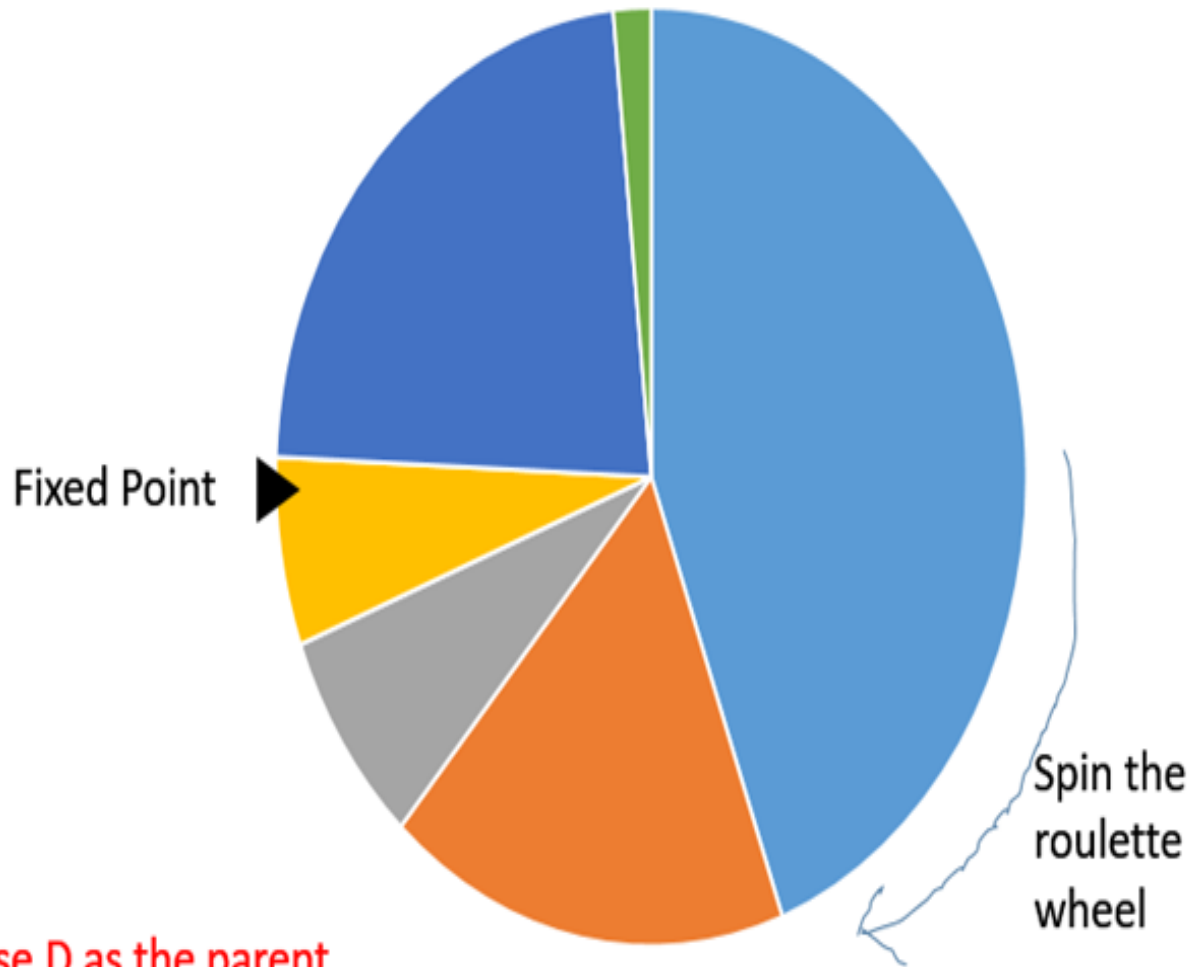
- Two type of selection scheme
 - **Proportionate based selection**
 - Pick out the individuals based upon their fitness values relative to the fitness of other individuals in the population
 - Selection pressure depends on the fitness
 - **Ordinal based selection**
 - Not based on their fitness value
 - Upon the rank within the population
 - Selection pressure is independent of the fitness



SELECTION

- Various selection methods are
 - **Roulette Wheel Selection**
 - **Random Selection**
 - **Rank Selection**
 - **Tournament Selection**
 - **Boltzmann Selection**
 - **Stochastic Universal Sampling**

Roulette Wheel Selection



Chromosome	Fitness Value
A	8.2
B	3.2
C	1.4
D	1.2
E	4.2
F	0.3

■ A ■ B ■ C ■ D ■ E ■ F

Arsha J K, AP, Computer Science and
Engineering, VJCET, Vazhakulam

Roulette Wheel Selection

- Consider a circular wheel.
- The wheel is divided into n pies, where n is the number of individuals in the population.
- Each individual gets a portion of the circle which is proportional to its fitness value.
- In a roulette wheel selection, the circular wheel is divided as described before.
- A fixed point is chosen on the wheel circumference as shown and the wheel is rotated.
- The region of the wheel which comes in front of the fixed point is chosen as the parent.
- For the second parent, the same process is repeated.

Roulette Wheel Selection

- It is clear that a fitter individual has a greater pie on the wheel and therefore a greater chance of landing in front of the fixed point when the wheel is rotated.
- Therefore, the probability of choosing an individual depends directly on its fitness.
- Implementation wise, we use the following steps –
 - Calculate S = the sum of a fitnesses.
 - Generate a random number between 0 and S .
 - Starting from the top of the population, keep adding the fitnesses to the partial sum P , till $P < S$.
 - The individual for which P exceeds S is the chosen individual.

Roulette Wheel Selection

- **Easier to implement**
- **But Noisy**
- **Depends on the variance of fitness in the population**

Random Selection

- Randomly selects a parent from the population
- A little more disruptive

Rank Selection

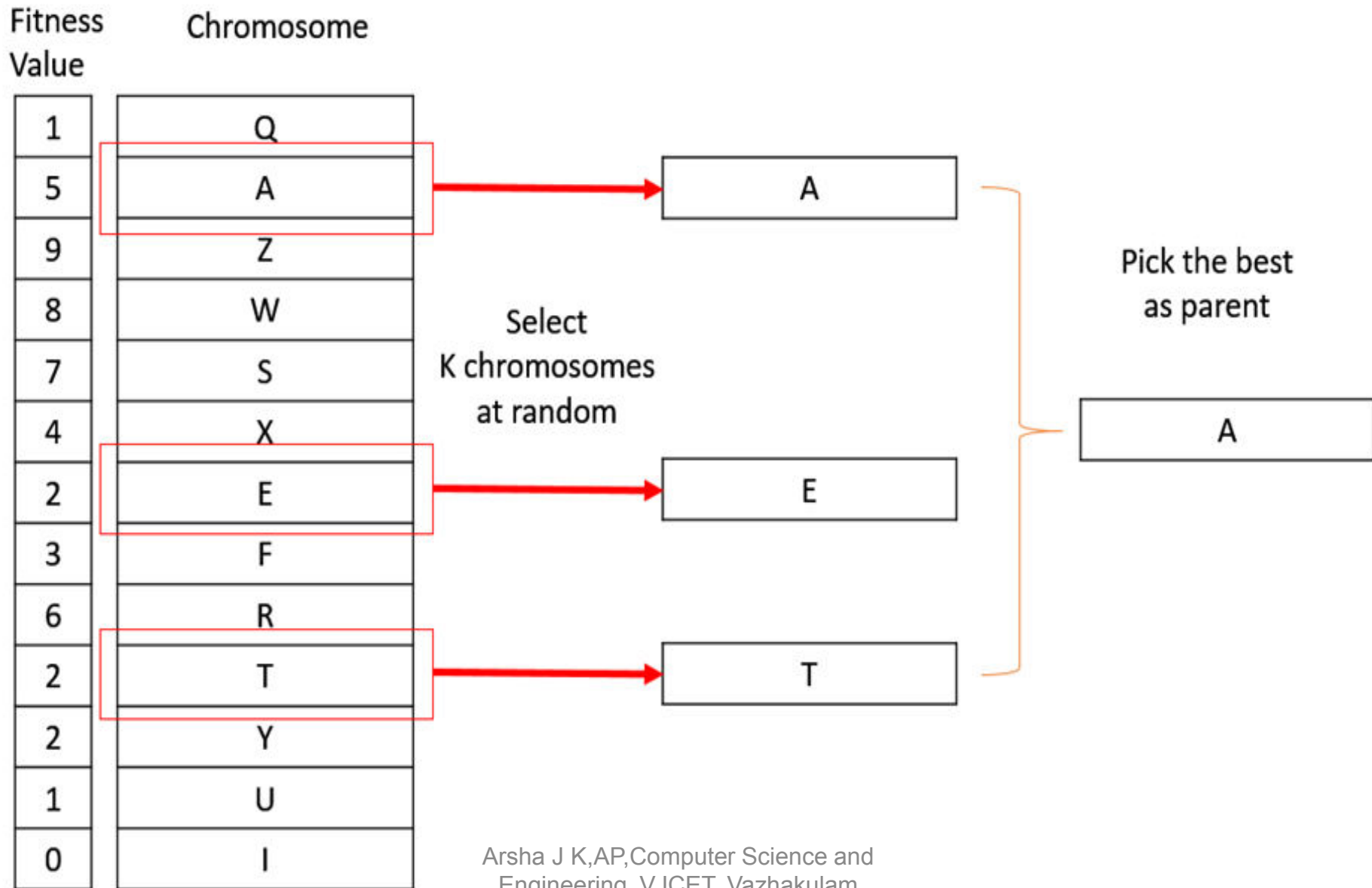
- Rank selection ranks the population and every chromosome receives fitness from the ranking
- The worst has fitness 1 and the best has fitness N
- It results in slow convergence but prevents too quick convergence
- It also keep up selection pressure when the fitness variance is low
- Preserve diversity

Chromosome	Fitness Value	Rank
A	8.1	1
B	8.0	4
C	8.05	2
D	7.95	6
E	8.02	3
F	7.99	5

Rank Selection

- **There are many ways for rank selection**
 - **Select a pair of individual at random. Generate a random number R between 0 and 1 . If $R < r$ use the first individual as parent. This is repeated to select the second parent. The value of r is a parameter to this method**
 - **Select two individuals at random. The individual with a highest evaluation become the parent. Repeat to find the second parent**

Tournament Selection



Tournament Selection

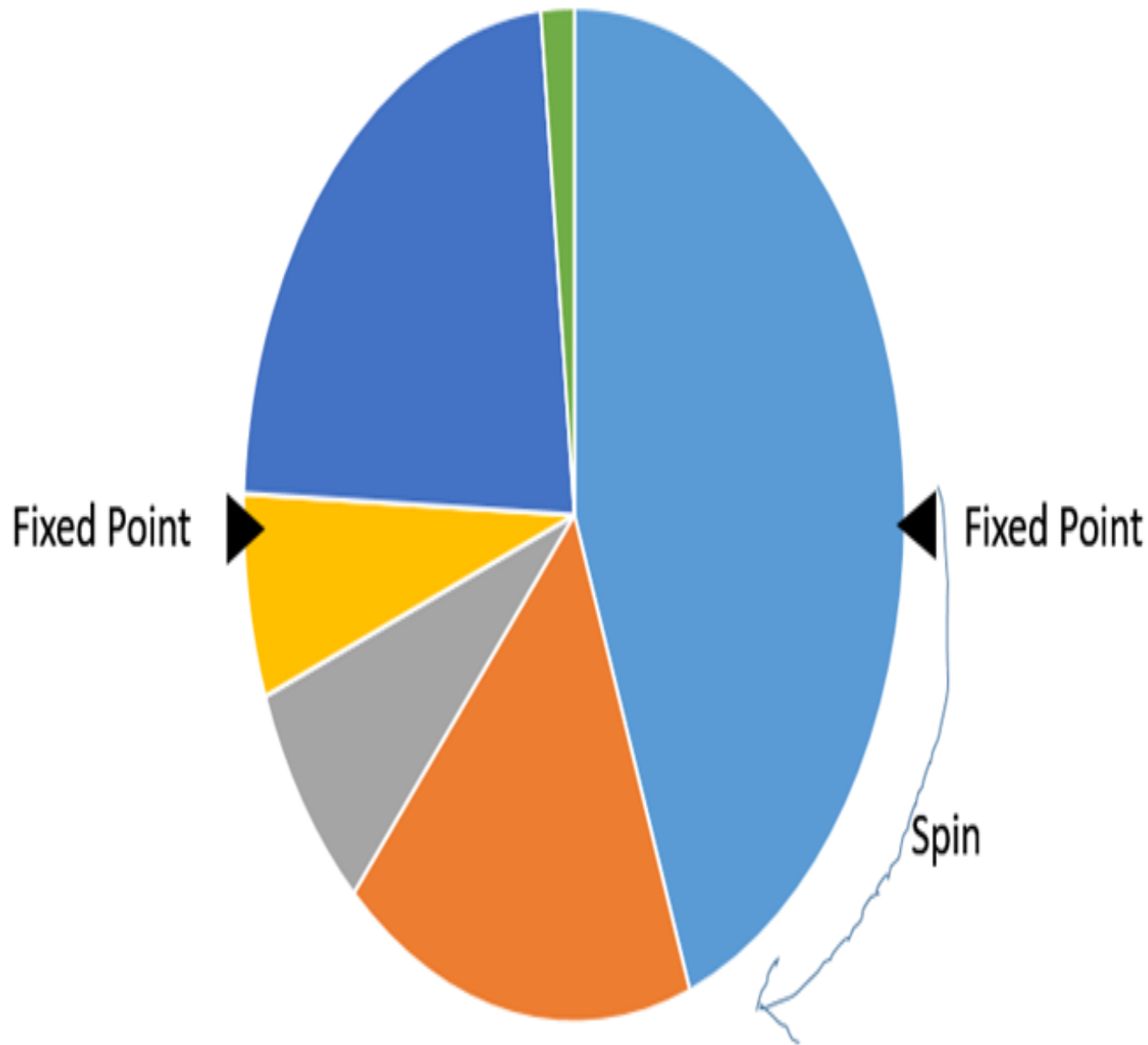
- In K-Way tournament selection, we select K individuals from the population at random and select the best out of these to become a parent.
- The same process is repeated for selecting the next parent.
- Tournament Selection is also extremely popular in literature as it can even work with negative fitness values.

Boltzmann Selection

- This method simulates the process of slow cooling of molten metal to achieve the minimum function value in a minimization problem
- Controlling a temperature like parameter introduced with the concept of Boltzmann probability distribution simulate the cooling phenomenon
- In Boltzmann selection, a continuously varying temperature controls the rate of selection according to a preset schedule
- The temperature start out high, which means that the selection pressure is low
- The temperature is gradually lowered, which gradually increase the selection pressure
- Thereby allowing the GA to narrow in more closely to the best part of the search space

Stochastic Universal Sampling

- **Stochastic Universal Sampling is quite similar to Roulette wheel selection, however instead of having just one fixed point, we have multiple fixed points as shown in the following image.**
- **Therefore, all the parents are chosen in just one spin of the wheel.**
- **Also, such a setup encourages the highly fit individuals to be chosen at least once.**



Chromosome	Fitness Value
A	8.2
B	3.2
C	1.4
D	1.2
E	4.2
F	0.3

CROSSOVER

- **Cross over is a process of taking two parent solution and producing from them a child**
- **After the selection process the population is enriched with better individuals**
- **Reproduction makes the clone of good strings but does not create new one**
- **Crossover operator is applied to the matting pool with a hope that it creates a better offspring**

CROSSOVER

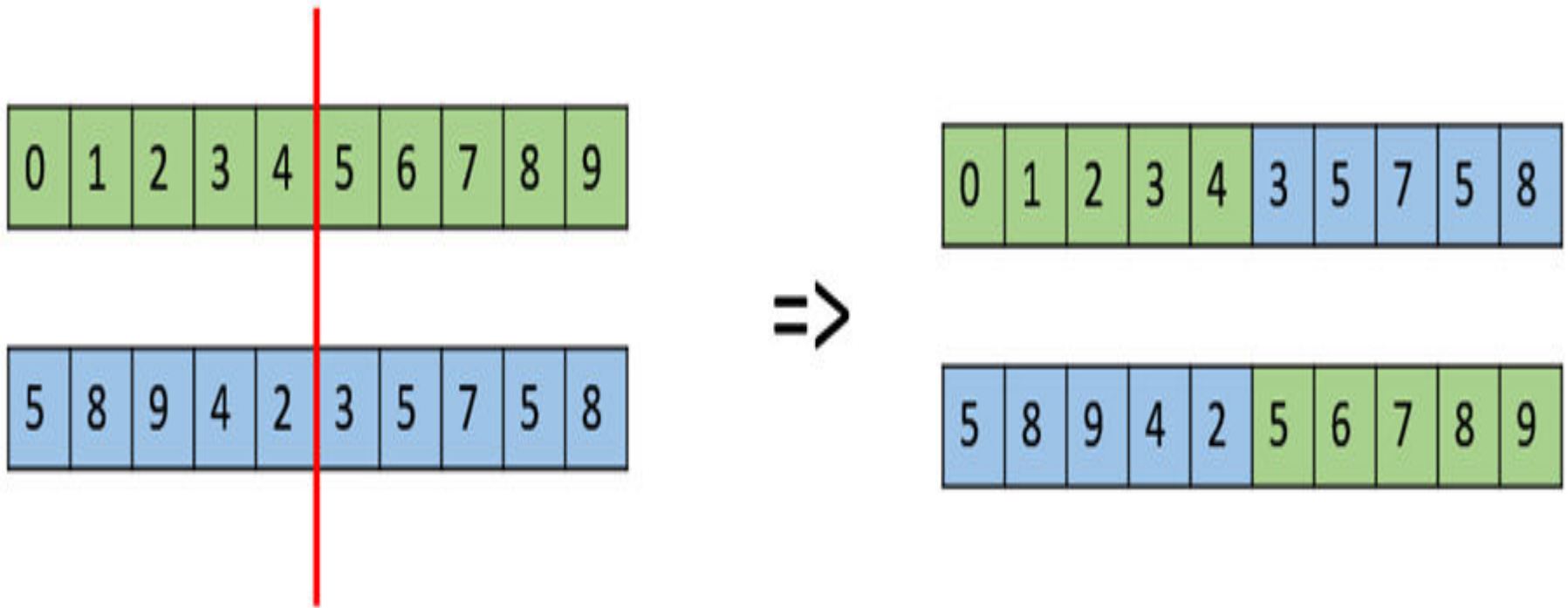
- **Cross over is a recombination operator that proceeds in three steps**
 - 1. The reproduction operator select at random a pair of 2 individual strings for the matting**
 - 2. A cross site is selected at random along the string length**
 - 3. Finally, the position values are swapped between the two strings following the cross site**

CROSSOVER

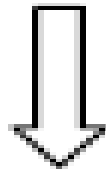
- **Various crossover techniques are**
 - **Single- point Crossover**
 - **Two- point crossover**
 - **Multipoint cross over/N point cross over**
 - **Uniform Crossover**
 - **Three parent Crossover**
 - **Crossover with reduced Surrogate**
 - **Shuffle crossover**
 - **Precedence Preservative Crossover**
 - **Ordered crossover**
 - **Partially matched crossover**

Single-Point Crossover

- In this one-point crossover, a random crossover point is selected and the tails of its two parents are swapped to get new off-springs.



Parent 1	1 0 1 1 0	0 1 0
Parent 2	1 0 1 0 1	1 1 1



Child 1	1 0 1 1 0	1 1 1
Child 2	1 0 1 0 1	0 1 0

parents

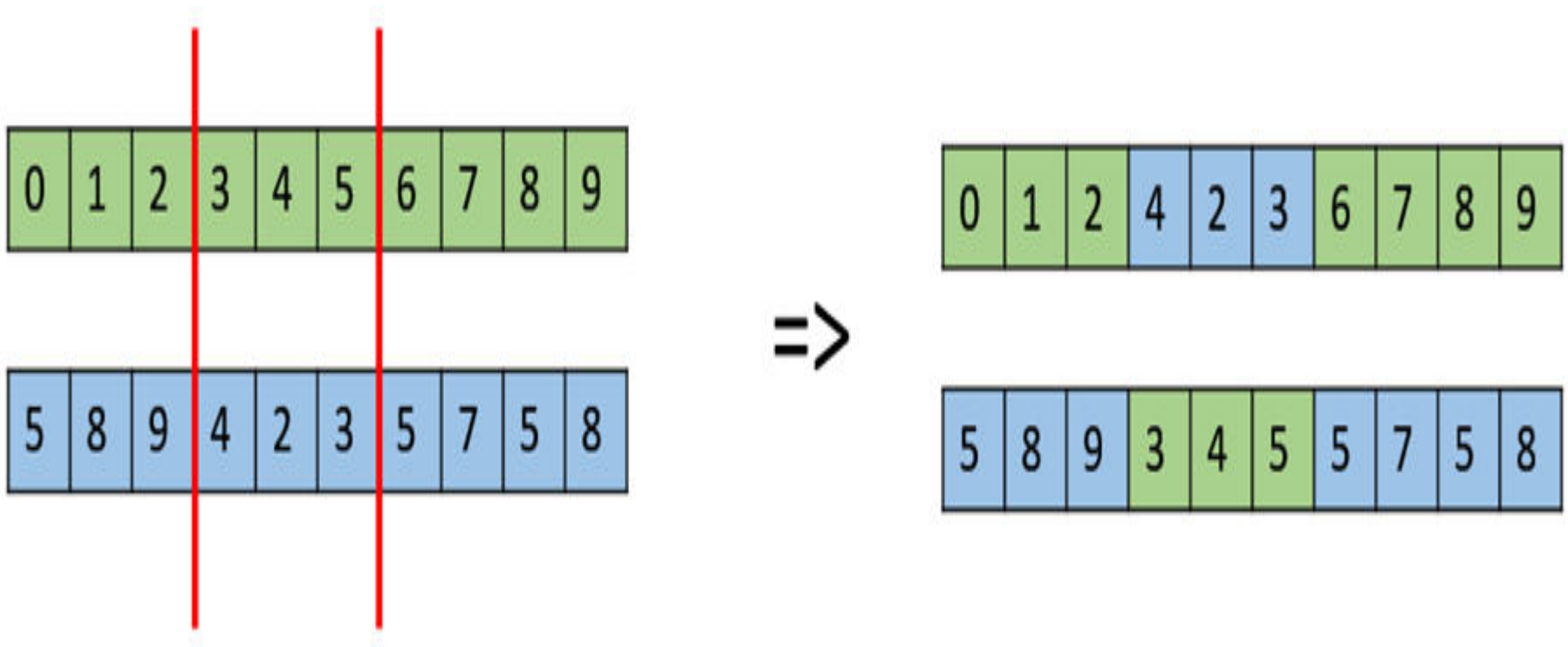
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

children

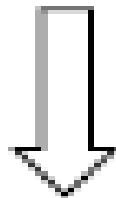
0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0

Two-Point Crossover

- Two crossover points are chosen and the content between these points are exchanged between two mated parents



Parent 1	1 1 0 1 1 0 1 0
Parent 2	0 1 1 0 1 1 0 0



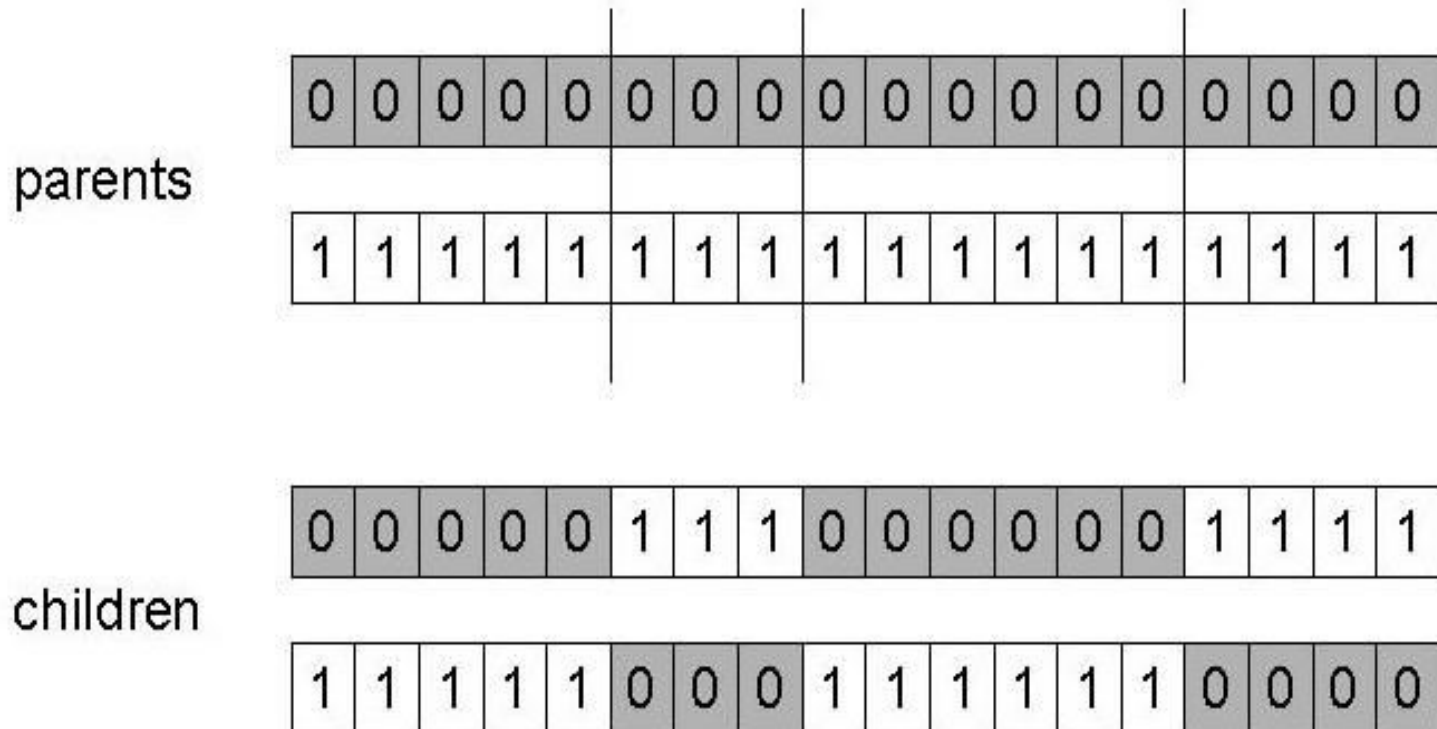
Child 1	1 1 0 0 1 1 1 0
Child 2	0 1 1 1 1 0 0 0

Two-Point Crossover

- But with one point cross over , the head and tail of one chromosome cannot be passed together to the offspring
- If both the head and tail of chromosome contain good genetic information, none of the offspring obtain directly with the one point crossover
- Using two point crossover one can overcome these drawbacks so it is better than one point cross over

Multi-Point Crossover/ (N-Point crossover)

- Multi point crossover is a generalization of the one-point crossover wherein alternating segments are swapped to get new off-springs.



Uniform Crossover

PARENT 1	1 0 1 1 0 0 1 1
PARENT 2	0 0 0 1 1 0 1 0
MASK	1 1 0 1 0 1 1 0
CHILD 1	1 0 0 1 1 0 1 0
CHILD 2	0 0 1 1 0 0 1 1

Uniform Crossover

- Each gene in the offspring is created by copying the corresponding gene from the one or other parent chosen according to the random generated binary crossover mask of same length as the chromosome
- Where there is a 1 in the crossover point, the gene is copied from the first parent
- Where there is a 0 in the mask , the gene is copied from the second parent
- A new crossover mask is generated for each parent pair
- Offspring therefore contain a mixture of gene from each parent
- The number of effective crossing point is not fixed , but will be the average of $L/2$ where L is the chromosome length

Uniform Crossover

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

5	1	9	4	4	5	5	7	5	9
---	---	---	---	---	---	---	---	---	---

⇒

5	8	9	4	2	3	5	7	5	8
---	---	---	---	---	---	---	---	---	---

0	8	2	3	2	3	6	7	8	8
---	---	---	---	---	---	---	---	---	---

Genetic Algorithm – Uniform Crossover

Generate uniformly random number.

X1 = **0** 1 1 0 0 0 **1** 0 1 0

X2 = **1** 1 0 0 0 0 **0** 1 1 1

Uniformly generated = **1** 0 0 0 0 0 **1** 0 0 0

As a result, the new population becomes,

X1 = **1** 1 1 0 0 0 **0** 0 1 0

X2 = **0** 1 0 0 0 0 **1** 1 1 1

Three- Parent Crossover

PARENT 1	1 1 0 1 0 0 0 1
PARENT 2	0 1 1 0 1 0 0 1
PARENT 3	0 1 1 0 1 1 0 0
CHILD	0 1 1 0 1 0 0 1

Three- Parent Crossover

- Three parents are randomly chosen
- Each bit of the first parent is compared with the bit of second parent.
- If both are same, the bit is taken for the offspring
- Otherwise bit from the third parent is taken for the offspring

Crossover with Reduced Surrogate

The reduced surrogate operator constrains crossover to always produce new individuals wherever possible.

This is implemented by restricting the location of crossover points such that crossover points only occur where gene values differ.

Shuffle Crossover

Shuffle crossover is related to uniform crossover. A single crossover position (as in single-point crossover) is selected.

But before the variables are exchanged, they are randomly shuffled in both parents.

After recombination, the variables in the offspring are unshuffled.

This removes positional bias as the variables are randomly reassigned each time crossover is performed.

Precedence Preservative Crossover (PPX)

- Developed for vehicle routing problems
- The operator passes on precedence relation of operations given in two parental permutations to one offspring at the same rate, while no new precedence relations are introduced

- 1. A vector of length Σ , sub $i=1$ to m_i , representing the number of operations involved in the problem, is randomly filled with elements of the set $\{1, 2\}$.**
- 2. This vector defines the order in which the operations are successively drawn from parent 1 and parent 2.**
- 3. We can also consider the parent and offspring permutations as lists, for which the operations 'append' and 'delete' are defined.**
- 4. First we start by initializing an empty offspring.**
- 5. The leftmost operation in one of the two parents is selected in accordance with the order of parents given in the vector.**
- 6. After an operation is selected it is deleted in both parents.**
- 7. Finally the selected operation is appended to the offspring.**
- 8. This step is repeated until both parents are empty and the offspring contains all operations involved.**

Precedence Preservative Crossover (PPX)

Parent permutation 1	A	B	C	D	E	F
Parent permutation 2	C	A	B	F	D	E
Select parent no. (1/2)	1	2	1	1	2	2
Offspring permutation	A	C	B	D	F	E

Ordered Crossover

- Ordered two-point crossover is used when the problem is of order based, for example in U-shaped assembly line balancing etc.
- Given two parent chromosomes, two random crossover points are selected partitioning them into a left, middle and right portion.
- The ordered two-point crossover behaves in the following way:
 - child 1 inherits its left and right section from parent 1, and its middle section is determined by the genes in the middle section of parent 1 in the order in which the values appear in parent 2.
- A similar process is applied to determine child 2.

Parent 1 : 4 2 1 3 6 5	Child 1 : 4 2 3 1 6 5
Parent 2 : 2 3 1 4 5 6	Child 2 : 2 3 4 1 5 6

Partially Matched Crossover (PMX)

- PMX can be applied usefully in the TSP.
- Indeed, TSP chromosomes are simply sequences of integers, where each integer represents a different city and the order represents the time at which a city is visited.
- Under this representation, known as permutation encoding, we are only interested in labels .
- It may be viewed as a crossover of permutations that guarantees that all positions are found exactly once in each offspring, i.e. both 3 offspring receive a full complement of genes, followed by the corresponding filling in of alleles from their parents.

Partially Matched Crossover (PMX)

- **PMX proceeds as follows:**
 1. **The two chromosomes are aligned.**
 2. **Two crossing sites are selected uniformly at random along the strings, defining a matching section**
 3. **The matching section is used to effect a cross through position by-position exchange operation**
 4. **Alleles are moved to their new positions in the offspring**

- ❑ Consider the two strings shown in Fig
- ❑ Where, the dots mark the selected cross points.
- ❑ The matching section defines the position-wise exchanges that must take place in both parents to produce the offspring.
- ❑ The exchanges are read from the matching section of one chromosome to that of the other.
- ❑ In the example, the numbers that exchange places are 5 and 2, 6 and 3, and 7 and 10.
- ❑ The resulting offspring are as shown

exchange places are 5 and 2, 6 and 3, and 7 and 10.

Name 9 8 4 . 5 6 7 . 1 3 2 1 0 Allele 1 0 1 . 0 0 1 . 1 1 0 0

Name 8 7 1 . 2 3 1 0 . 9 5 4 6 Allele 1 1 1 . 0 1 1 . 1 1 0 1

Name 9 8 4 . 2 3 1 0 . 1 6 5 7 Allele 1 0 1 . 0 1 0 . 1 0 0 1

Name 8 1 0 1 . 5 6 7 . 9 2 4 3 Allele 1 1 1 . 1 1 1 . 1 0 0 1

Parent A

4 8 7 | 3 6 5 | 1 10 9 2

Parent B

3 1 4 | 2 7 9 | 10 8 6 5

Parent A 4 8 7 | 3 6 5 | 1 10 9 2

Parent B 3 1 4 | 2 7 9 | 10 8 6 5

Child A 4 8 6 2 7 9 1 10 5 3

Child B 2 1 4 3 6 5 10 8 7 9

Crossover Probability

The basic parameter in crossover technique is the crossover probability (P_c).

Crossover probability is a parameter to describe how often crossover will be performed.

- If there is no crossover, offspring are exact copies of parents. If there is crossover, offspring are made from parts of both parent's chromosome.
- If crossover probability is 100%, then all offspring are made by crossover.
- If it is 0%, whole new generation is made from exact copies of chromosomes from old population (but this does not mean that the new generation is the same!).

Mutation

- After crossover, the strings are subjected to mutation. Mutation prevents the algorithm to be trapped in a local minimum.
- Mutation plays the role of recovering the lost genetic materials as well as for randomly disturbing genetic information.
- It is an insurance policy against the irreversible loss of genetic material. Mutation has traditionally considered as a simple search operator.
- If crossover is supposed to exploit the current solution to find better ones, mutation is supposed to help for the exploration of the whole search space.
- Mutation of a bit involves flipping a bit, changing 0 to 1 and vice-versa.

Mutation Methods

- **Flipping**
- **Interchanging**
- **Reversing**

Flipping

Flipping of a bit involves changing 0 to 1 and 1 to 0 based on a mutation chromosome generated. A parent is considered and a mutation chromosome is randomly generated.

Parent	1 0 1 1 0 1 0 1
Mutation chromosome	1 0 0 0 1 0 0 1
Child	0 0 1 1 1 1 0 0



Interchanging

Two random positions of the string are chosen and the bits corresponding to those positions are interchanged.

Parent	1 0 1 1 0 1 0 1
Child	1 1 1 1 0 0 0 1

1	2	3	4	5	6	7	8	9	0
---	---	---	---	---	---	---	---	---	---

=>

1	6	3	4	5	2	7	8	9	0
---	---	---	---	---	---	---	---	---	---

Reversing

A random position is chosen and the bits next to that position are reversed and child chromosome is produced.

Parent	1 0 1 1 0 1 0 1
Child	1 0 1 1 0 1 1 0

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

=>

0	1	6	5	4	3	2	7	8	9
---	---	---	---	---	---	---	---	---	---

Mutation Probability

- The important parameter in the mutation technique is the mutation probability (P_m).
- The mutation probability decides how often parts of chromosome will be mutated.
- If there is no mutation, offspring are generated immediately after crossover (or directly copied) without any change.
- If mutation is performed, one or more parts of a chromosome are changed. If mutation probability is 100%, whole chromosome is changed.
- if it is 0%, nothing is changed.
- Mutation generally prevents the GA from falling into local extremes.
- Mutation should not occur very often, because then GA will in fact change to random search.

Replacement

- Replacement is the last stage of any breeding cycle. Two parents are drawn from a fixed size population, they breed two children, but not all four can return to the population, so two must be replaced. The technique used to decide which individual stay in a population and which are replaced in on a par with the selection in influencing convergence. Basically, there are two kinds of methods for maintaining the population; *generational updates* and *steady state updates*.

Generational update scheme

The basic generational update scheme consists in producing N children from a population of size N to form the population at the next time step (generation), and this new population of children completely replaces the parent selection. Clearly this kind of update implies that an individual can only reproduce with individuals from the same generation.

Steady state update

In a steady state update, new individuals are inserted in the population as soon as they are created, as opposed to the generational update where an entire new generation is produced at each time step. The insertion of a new individual usually necessitates the replacement of another population member. The individual to be deleted can be chosen as the worst member of the population.

Random Replacement

The children replace two randomly chosen individuals in the population. The parents are also candidates for selection. This can be useful for continuing the search in small populations, since weak individuals can be introduced into the population.

Weak Parent Replacement

- In weak parent replacement, a weaker parent is replaced by a strong child.
- With the four individuals only the fittest two, parent or child, return to population.

Both Parents Replacement

Both parents replacement is simple. The child replaces the parent. In this case, each individual only gets to breed once. As a result, the population and genetic material moves around but leads to a problem when combined with a selection technique that strongly favors fit parents: the fit breed and then are disposed of.

Search Termination (Convergence Criteria)

- **Maximum generations**–The genetic algorithm stops when the specified number of generation's have evolved.
- **Elapsed time**–The genetic process will end when a specified time has elapsed.

Note: If the maximum number of generation has been reached before the specified time has elapsed, the process will end.

- **No change in fitness**–The genetic process will end if there is no change to the population's best fitness for a specified number of generations.

Search Termination (Convergence Criteria)

- **Stall generations**—The algorithm stops if there is no improvement in the objective function for a sequence of consecutive generations of length **Stall generations**.
- **Stall time limit**—The algorithm stops if there is no improvement in the objective function during an interval of time in seconds equal to **Stall time limit**.

Stopping condition for GA flow

- **Best Individual:** A best individual convergence criterion stops the search once the minimum fitness in the population drops below the convergence value. This brings the search to a faster conclusion guaranteeing at least one good solution.
- **Worst individual:** Worst individual terminates the search when the least fit individuals in the population have fitness less than the convergence criteria. This guarantees the entire population to be of minimum standard, although the best individual may not be significantly better than the worst. In this case, a stringent convergence value may never be met, in which case the search will terminate after the maximum has been exceeded.

Stopping condition for GA flow

- ***Sum of Fitness:*** In this termination scheme, the search is considered to have satisfaction converged when the sum of the fitness in the entire population is less than or equal to the convergence value in the population record.
- ***Median Fitness:*** Here at least half of the individuals will be better than or equal to the convergence value, which should give a good range of solutions to choose from.

Why do Genetic Algorithms Work?

The search heuristics of GA are based upon Holland's scheme theorem.

A schema is defined as templates for describing a subset of chromosomes with similar sections. The schemata consist of bits 0, 1 and meta-character.

The template is a suitable way of describing similarities among Patterns in the chromosomes Holland derived an expression that predicts the number of copies of a particular schema would have in the next generation after undergoing exploitation, crossover and mutation.

Why do Genetic Algorithms Work?

It should be noted that particularly good schemata will propagate in future generations. Thus, schema that are low-order, well defined and have above average fitness are preferred and are termed building blocks.

This leads to a building block principle of GA: low order, well-defined, average fitness schemata will combine through crossover to form high order, above average fitness schemata.

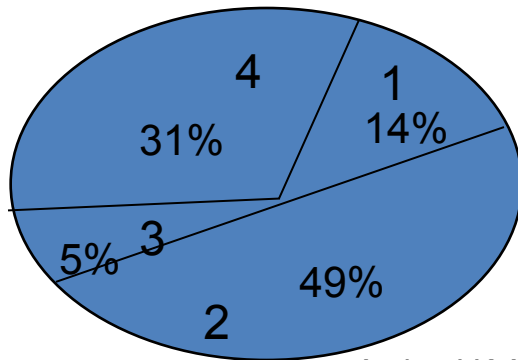
Since GAs process many schemata in a given generation they are said to have the property of implicit parallelism.

Genetic Algorithm – An Example

- Simple problem: $\max x^2$ over $\{0, 1, \dots, 31\}$
- GA approach:
 - Representation: binary code, e.g. $01101 \leftrightarrow 13$
 - Population size: 4
 - 1-point crossover, bitwise mutation
 - Roulette wheel selection
 - Random initialisation
- One generational cycle performed manually is shown here.

Example : Selection

String no.	Initial population	x Value	Fitness $f(x) = x^2$	$Prob_i$	Expected count	Actual count
1	0 1 1 0 1	13	169	0.14	0.58	1
2	1 1 0 0 0	24	576	0.49	1.97	2
3	0 1 0 0 0	8	64	0.06	0.22	0
4	1 0 0 1 1	19	361	0.31	1.23	1
Sum			1170	1.00	4.00	4
Average			293	0.25	1.00	1
Max			576	0.49	1.97	2



Example : Crossover

String no.	Mating pool	Crossover point	Offspring after xover	x Value	Fitness $f(x) = x^2$
1	0 1 1 0 1	4	0 1 1 0 0	12	144
2	1 1 0 0 0	4	1 1 0 0 1	25	625
2	1 1 0 0 0	2	1 1 0 1 1	27	729
4	1 0 0 1 1	2	1 0 0 0 0	16	256
Sum					1754
Average					439
Max					729

Example : Mutation

String no.	Offspring after xover	Offspring after mutation	x Value	Fitness $f(x) = x^2$
1	0 1 1 0 0	1 1 1 0 0	26	676
2	1 1 0 0 1	1 1 0 0 1	25	625
2	1 1 0 1 1	1 1 0 1 1	27	729
4	1 0 0 0 0	1 0 1 0 0	18	324
Sum				2354
Average				588.5
Max				729

Simple Genetic Algorithm

- Has been subject of many (early) studies
 - still often used as benchmark for novel Gas.
- Shows many shortcomings, e.g.
 - Representation is too restrictive.
 - Mutation & crossovers only applicable for bit-string & integer representations.
 - Selection mechanism sensitive for converging populations with close fitness values.

Comparison of GA with Traditional Optimization Techniques

- GA works with the coding of solution set and not with the solution itself.
- GA uses population of solutions rather than a single solution for searching.
- GA uses fitness function for evaluation rather the derivatives.
- GA uses probabilistic transition and not deterministic rules.

References

- Holland, J. (1992), *Adaptation in natural and artificial systems*, 2nd Ed. Cambridge: MIT Press.
- Davis, L. (Ed.) (1991), *Handbook of genetic algorithms*. New York: Van Nostrand Reinhold.
- Goldberg, D. (1989), *Genetic algorithms in search, optimization and machine learning*. Addison-Wesley.
- Fogel, D. (1995), *Evolutionary computation: Towards a new philosophy of machine intelligence*. Piscataway: IEEE Press.
- Bäck, T., Hammel, U., and Schwefel, H. (1997), 'Evolutionary computation: Comments on the history and the current state', *IEEE Trans. On Evol. Comp.* 1, (1)

Online Resources

- <http://www.spectroscopynow.com>
- <http://www.cs.bris.ac.uk/~colin/evollect1/evollect0/index.htm>
- IlliGAL (<http://www-illigal.ge.uiuc.edu/index.php3>)
- GAlib (<http://lancet.mit.edu/ga/>)

HYBRID SOFT COMPUTING TECHNIQUES

Arsha J K, AP, Computer Science and
Engineering, VJCET, Vazhakulam

GENERAL NEURO HYBRID SYSTEMS

Definition:

A neuro-genetic hybrid or a genetic_neuro-hybrid system is one in which a neural network employs a genetic algorithm to optimize its structural parameters that define its architecture.

Properties of genetic neuro-hybrid systems:

1. The parameters of neural networks are encoded by genetic algorithms as a string of properties of the network, that is, chromosomes. A large population of chromosomes is generated, which represent the many possible parameter sets for the given neural network.
2. Genetic Algorithm _Neural Network, or GANN, has the ability to locate the neighborhood of the optimal solution quickly, compared to other conventional search strategies.

Drawbacks are :

1. Large amount of memory required for handling and manipulation of chromosomes for a given network
2. Scalability as the network becomes large

BLOCK DIAGRAM OF GENETIC NEURO HYBRID SYSTEMS

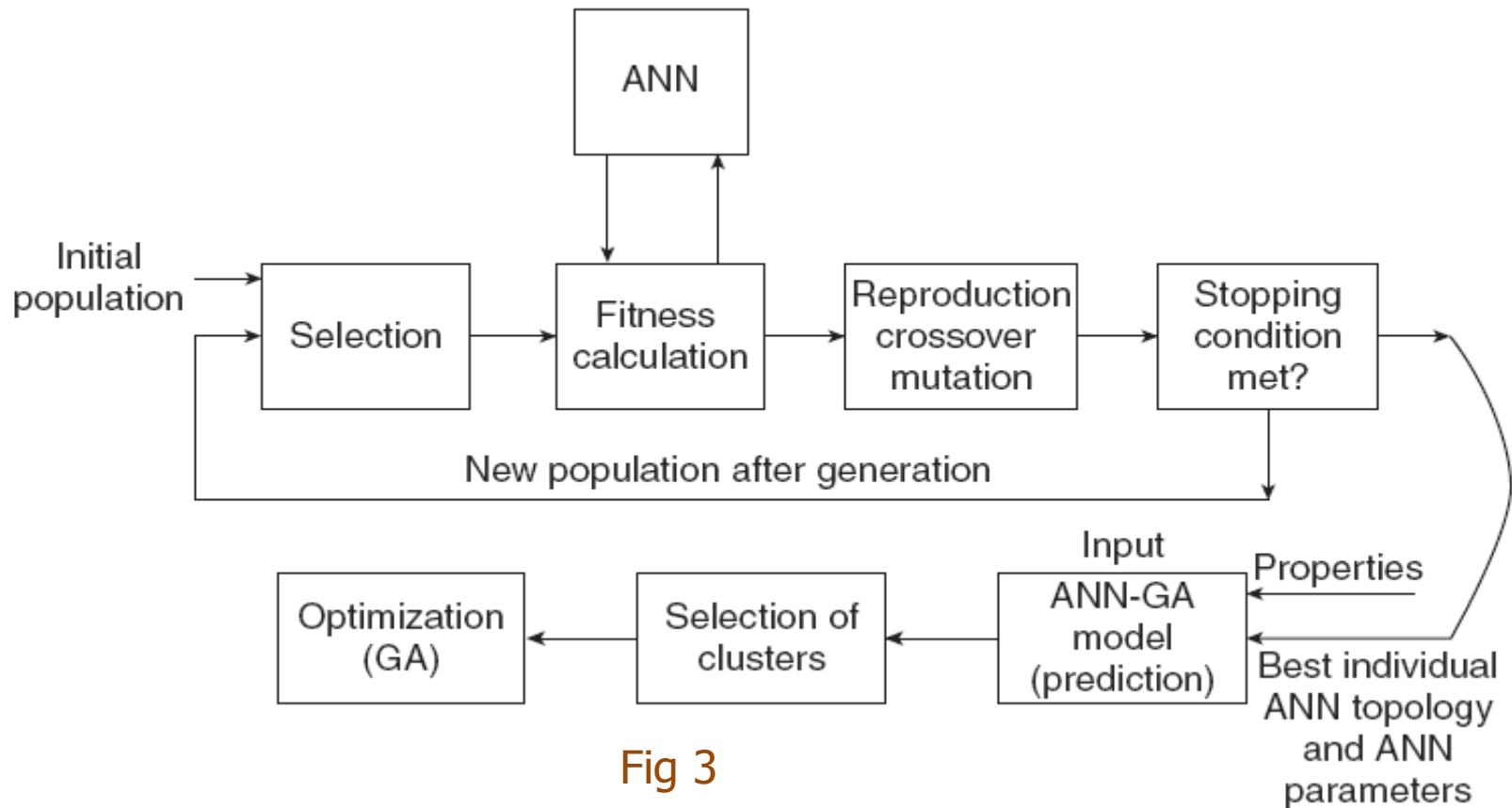


Fig 3

ADVANTAGES OF GENETIC NEURO HYBRID SYSTEMS

The various advantages of neuro-genetic hybrid are as follows:

- GA performs optimization of neural network parameters with simplicity, ease of operation, minimal requirements and global perspective.
- GA helps to find out complex structure of ANN for given input and the output data set by using its learning rule as a fitness function.
- Hybrid approach ensembles a powerful model that could significantly improve the predictability of the system under construction.

GENETIC FUZZY HYBRID SYSTEMS

The hybridization of genetic algorithm and fuzzy logic can be performed in the following two ways:

1. By the use of fuzzy logic based techniques for improving genetic algorithm behavior and modeling GA components. This is called fuzzy genetic algorithms (FGAs).
2. By the application of genetic algorithms in various optimization and search problems involving fuzzy systems.

ADVANTAGES OF GENETIC FUZZY HYBRID SYSTEMS

GAs allow us to represent different kinds of structures, such as weights, features together with rule parameters, etc., allowing us to code multiple models of knowledge representation. This provides a wide variety of approaches where it is necessary to design specific genetic components for evolving a specific representation.

Genetic algorithm efficiently optimizes the rules, membership functions, DB and KB of fuzzy systems. The methodology adopted is simple and the fittest individual is identified during the process.

SUMMARY

This chapter has given an overview on Hybrid Soft Computing Techniques